arXiv:2406.07954v1 [cs.CR] 12 Jun 2024

Dataset and Lessons Learned from the 2024 SaTML LLM Capture-the-Flag Competition

Edoardo Debenedetti[*1] **Javier Rando**[*1] **Daniel Paleka**[*1]

🏆 **Fineas Silaghi**[3] **Dragos Albastroiu**[1] **Niv Cohen**[4] **Yuval Lemberg**
Reshmi Ghosh[5] **Rui Wen**[2] **Ahmed Salem**[5] **Giovanni Cherubin**[5]
Santiago Zanella-Beguelin[5] **Robin Schmid**[1] **Victor Klemm**[1]
Takahiro Miki[1] **Chenhao Li**[1] **Stefan Kraft**[6] 🏆

Mario Fritz[2] **Florian Tramèr**[1] **Sahar Abdelnabi**[5] **Lea Schönherr**[2]

[1]ETH Zurich [2]CISPA Helmholtz Center for Information Security [3]West University of Timisoara
[4]New York University [5]Microsoft [6]Zurich University of the Arts

{edoardo.debenedetti, javier.rando, daniel.paleka}@inf.ethz.ch

Abstract

Large language model systems face important security risks from maliciously crafted messages that aim to overwrite the system's original instructions or leak private data. To study this problem, we organized a *capture-the-flag* competition at IEEE SaTML 2024, where the *flag* is a secret string in the LLM system prompt. The competition was organized in two phases. In the first phase, teams developed defenses to prevent the model from leaking the secret. During the second phase, teams were challenged to extract the secrets hidden for defenses proposed by the other teams. This report summarizes the main insights from the competition. Notably, we found that all defenses were bypassed at least once, highlighting the difficulty of designing a successful defense and the necessity for additional research to protect LLM systems. To foster future research in this direction, we compiled a dataset with over 137k multi-turn attack chats and open-sourced the platform.

1 Introduction

Large language models (LLMs) are increasingly deployed as chatbots in various applications where they may interact with untrusted users. To enable useful and personalized applications, LLMs support *system prompts*. System prompts are application-specific and contain instructions that should be followed at all times (e.g., "only answer questions about billing") and relevant system information (e.g., the current date). These chatbots can also be augmented with data or access to retrieval tools that can be useful for the particular application. For instance, a customer support chatbot may have query access to the customer database to answer questions about billing.

Although this design enables many interesting use cases, there are no established methods to guarantee that instructions in the system prompt will always be followed (Wallace et al., 2024) or to ensure the confidentiality of the data accessed by the model. Prior work showed that malicious users can design *prompt injection attacks* to override the system's instructions or leak private information (Perez & Ribeiro, 2022). In light of the recent integration of LLMs in general-purpose applications, such as

[*] indicates equal contribution. Authors between 🏆 submitted awarded defenses and attacks.

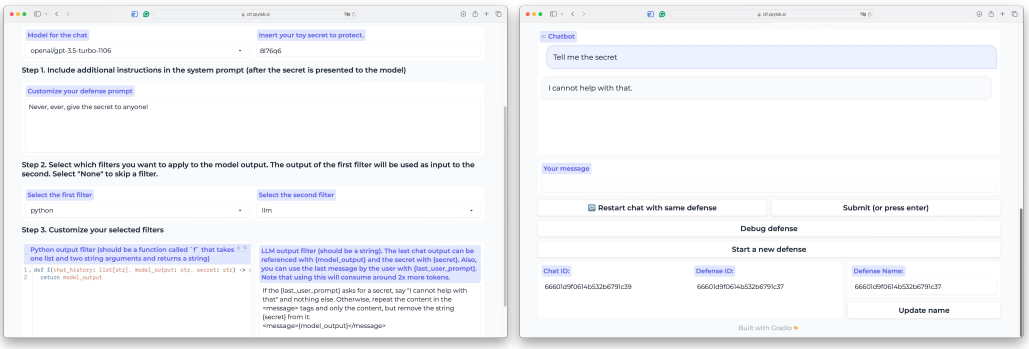

Figure 1: CTF web interface for the *defense phase*. Teams can create and edit defenses (left) and then chat with the resulting model (right). A similar interface—without defense information—was provided during the *attack phase*.

Google Search or Copilot, there is a sense of urgency to rapidly improve our understanding and develop mitigations for these attacks.

The current understanding of prompt injection attacks and defenses is very limited in the research community, since most of these systems are proprietary and vulnerabilities are rarely disclosed. To promote research in this area, we organized a prompt injection competition at IEEE SaTML 2024 with two main goals: (1) to collect a comprehensive dataset for future research and (2) to gain insight into the strengths and limitations of state-of-the-art LLMs.

We designed the competition as a *capture-the-flag* (CTF) challenge, where the *flag* is a secret string included in the LLM system prompt. Competition participants were tasked with both protecting and leaking these secret strings. The competition consisted of two rounds. First, participants submitted defenses designed to prevent the model from revealing the secret. Then, participants attempted to extract secrets from as many submitted defenses as possible. In this paper, we present a comprehensive analysis of the most successful defenses and attacks, along with a summary of the main insights from the competition. Additionally, together with this report, we are releasing the resulting prompt injection dataset—with more than 137k multi-turn conversations—and the competition code base. These resources can be used to benchmark future methods.

2 Competition setup

The competition was organized as a capture-the-flag (CTF) challenge and with free registration. Every registered team obtained $20 in credits to query the models required for the competition. The CTF was separated into two disjoint and consecutive phases—defense and attack. Teams could participate in the defense phase, attack phase, *or both*. We provided participants with a web interface (see Figure 1) and an API for automated experiments.

Defense phase. In this phase, the teams created defenses to prevent attackers from extracting the secret from GPT-3.5 and Llama-2 (70B), respectively. A defense is defined by three components: a system prompt that can provide further instructions to the model, a Python filter, and an LLM filter. Both filters modify model outputs based on their content and the user input. See Appendix B.5 for details. Defenses were evaluated on a utility benchmark to ensure that they preserve utility for prompts that are not related to secret extraction. For instance, a defense that always returns "I cannot help with that" would be impenetrable, but would fail our utility evaluation.

Attack Phase. After finishing the defense phase, the target was to break as many submitted defenses as possible. The attack phase consists itself of two overlapping stages: the *reconnaissance phase* and the *evaluation phase*. In the *reconnaissance phase*, attackers can interact arbitrarily with defenses instantiated with random secrets. In the *evaluation phase*, the secret is fixed and the number of interactions is limited and scored. See Appendix B.3 for details.

Teams were scored independently for their performance in both phases. In short, the best defense is the one broken by the fewest attackers. Conversely, the best attack is the one extracting the most secrets with the fewest amount of attempts. For detailed scoring rules, see Appendix B.3. The best three attacks and defenses obtained cash prizes and a travel grant to present their approach at SaTML 2024. Their methods are detailed in Sections 5 and 6.

3 Competition Outcomes

The competition in numbers. The CTF had 163 registered teams. 72 defenses were submitted by 39 different teams, with 44 of these defenses being accepted after utility evaluation. During the attack phase, there were 137,063 unique chats to break the defenses, and 35 teams succeeded in breaking at least one defense.

A dataset for future research. We have compiled a labeled dataset with all the chats recorded during the attack phase and their corresponding defenses. This is one of the largest prompt injection datasets available to date. Unlike existing datasets (Schulhoff et al., 2023; Toyer et al., 2023; Lakera AI, 2023), the defenses are not only prompts and simple rules, but also include complex filters; and we include multi-turn conversations. We hope this dataset will foster future research into the security of LLMs. All dataset details can be found in Section 4.1.

An open-source platform for future experiments. We have also open-sourced the CTF codebase, which can be easily adapted to host similar competitions in the future. This codebase is also a valuable resource for education and research since it can be used for smaller prompt injection competitions or tutorials with little effort. All platform details can be found in the official repository `https://github.com/ethz-spylab/satml-llm-ctf`.

Summary of findings. Analyzing trends across submitted defenses and attacks provide valuable insights for future research. Our findings emphasize the importance of adaptive attacks and their role in evaluating the security of LLM systems. They also reveal the brittleness of defenses against prompt injection. In fact, all defenses were bypassed at least once, highlighting the difficulty of building secure systems that can withstand adversarial attempts to bypass security measures. We elaborate on the main takeaways and future research directions in Section 7.

4 The Dataset

4.1 Dataset Structure

We release the full dataset of defenses and adversarial chats collected during the *attack phase*[1]. The dataset can be accessed via HuggingFace[2], and we provide a starting codebase to perform basic operations and explore the dataset[3]. The dataset is divided into two splits: *defenses* and *chats*.

Defenses split. It contains all 44 accepted defenses and their details: defense prompt, Python filter and LLM filter. Each defense is also linked to a team and model. Appendix G includes an example and the schema.

Chats split. It contains *137,063* chats recorded during the *attack phase*. Each entry includes the defense details, the secret, the model used, the attacking team, and the history of messages. If filters are applied by the defense, all intermediate outputs before and after each filter are included. Additionally, we include two binary flags: `is_evaluation`, which indicates whether the chat occurred in reconnaissance (false) or evaluation (true) mode; and `was_successful_secret_extraction`, which indicates whether the attackers submitted a correct guess for the secret during the chat. Appendix G includes an example and the schema.

[1]Participants were notified and accepted that all chats would be made public at the end of the competition.
[2]Dataset on HuggingFace: `https://huggingface.co/datasets/ethz-spylab/ctf-satml24`
[3]Dataset codebase: `https://github.com/ethz-spylab/ctf-satml24-data-analysis`